Image Filtering

Fundamentals of Data Science ‘24

Luca Scofano

SAPIENZA
UNIVERSITÀ DI ROMA

An image is a **function** that maps **locations** to **pixels**

$$\boxed{f(\boxed{x, y})} = \text{pixel}$$

This is an example using an **RGB** image

$$f(x, y) = \begin{bmatrix} \boxed{r(x, y)} \\ g(x, y) \\ b(x, y) \end{bmatrix}$$

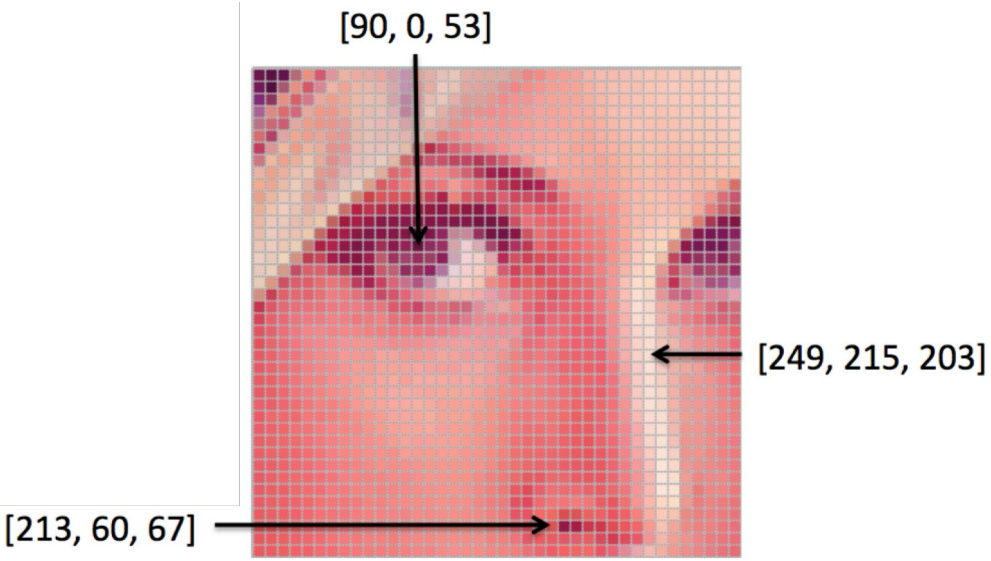
[0, 255] which equals to 2^8

What is the number of **combinations** a single pixel can have?

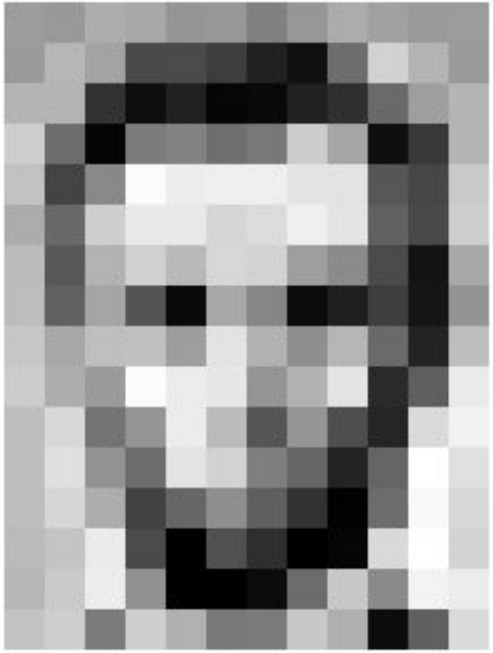
$$f(x,y) = \begin{bmatrix} \boxed{r(x,y)} \\ \boxed{g(x,y)} \\ \boxed{b(x,y)} \end{bmatrix}$$

256*256*256 = 16,777,216

What is the number of **combinations** a single pixel can have?



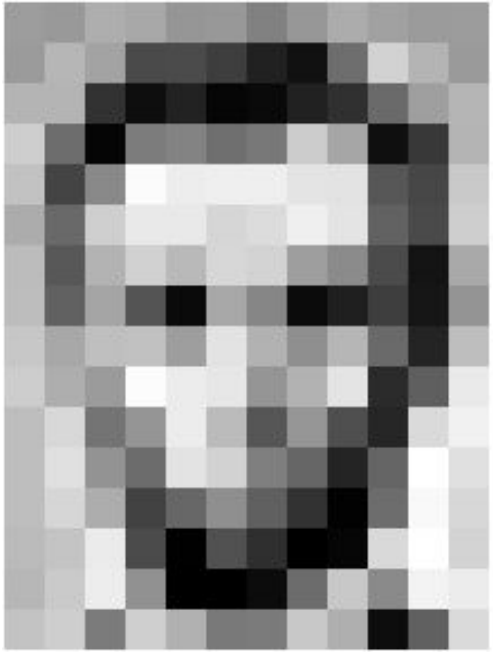
A more intuitive example (**Greyscale**)



What are the values a single pixel can have?

256

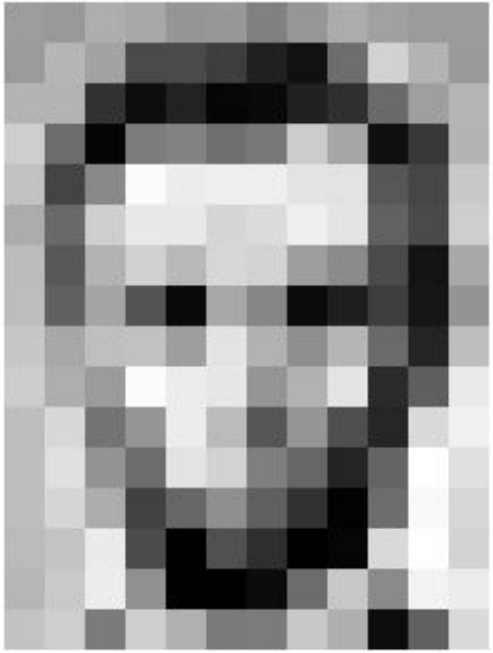
A more intuitive example (**Greyscale**)



=

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

A more intuitive example (**Greyscale**)



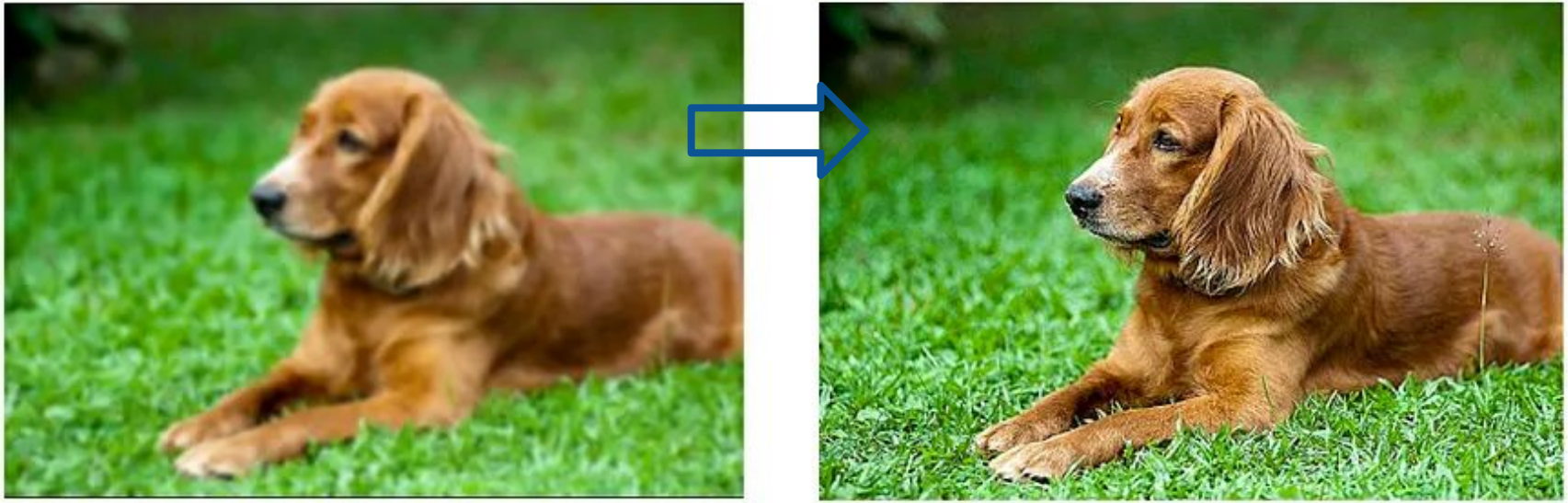
157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

Why do we need Image Filtering?

Image **Deblurring**

Why do we need Image Filtering?



How do we define a **filter**?

$$\boxed{f(\boxed{x, y})}$$

An **image** is a **function** that maps **locations** to **pixels**

$$h(f(\boxed{x, y}))$$

A **filter** is a **function** applied to the **pixels** of an image

Why do we need Image Filtering?

- Filtering for enhancement **improve contrast**
- Filtering for smoothing **removes noise**
- Filtering for **template matching** detects known patterns

Convolutions

- **Feature Extraction:** Convolutions enable the detection of local patterns. In deep learning, convolutional layers automatically learn these features at different levels of abstraction.

- **Localized Operations:** Convolutions operate on small, localized regions (called kernels or filters) of an input image or signal, making them efficient for tasks like smoothing, sharpening, or detecting gradients.

- **Shift Invariance:** Convolutional operations preserve the spatial structure of the input data, allowing the model to detect features regardless of their location within the image. This is crucial for tasks like object recognition.

2D-Convolutions

Which mathematical operations do we perform?

0	-1	0
-1	5	-1
0	-1	0

Kernel

1	3	2	0	1
2	1	0	1	3
1	3	2	1	1
2	0	2	3	2
1	3	1	2	1

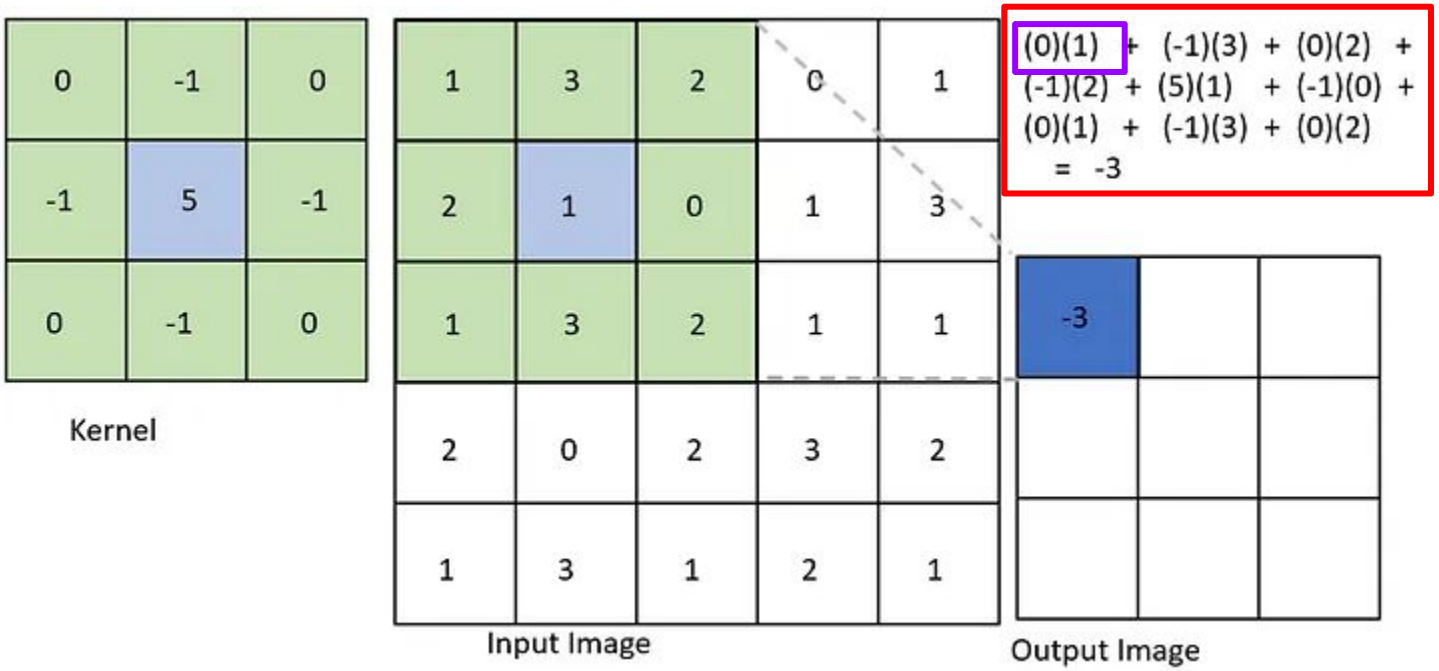
Input Image

2D-Convolutions

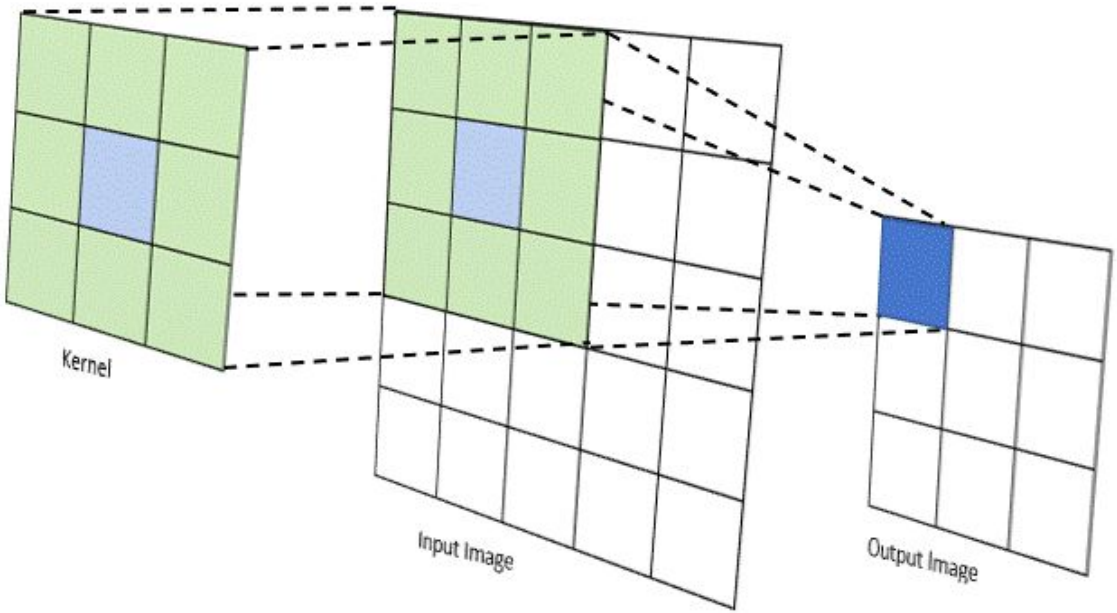
Element-Wise Matrix Multiplication

+

Summation

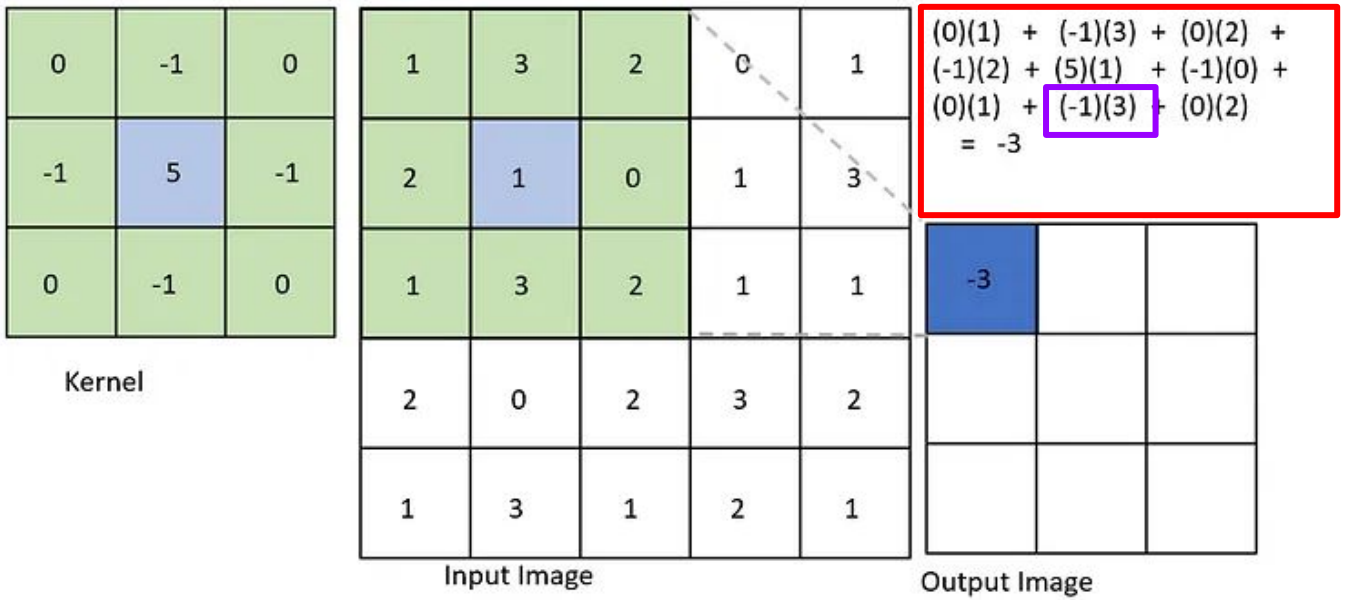


2D-Convolutions



Some Notation

$$\boxed{y(n,m)} = \boxed{\sum_k \sum_l \boxed{x(k,l)k(n-k,m-l))}}$$



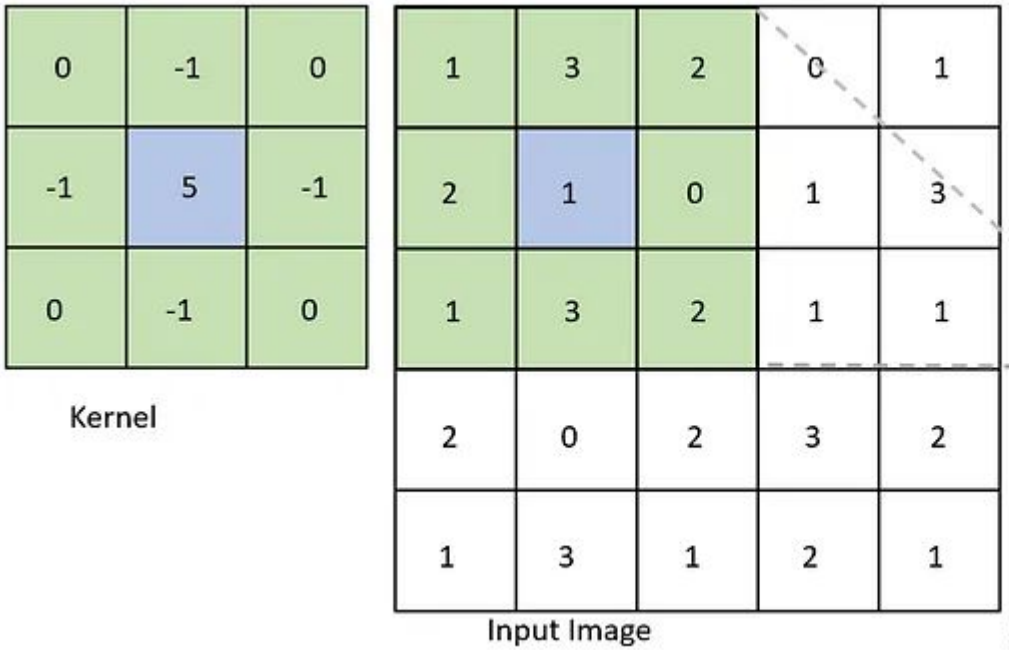
Beware ⚠

$$y(n,m) = \sum_k \sum_l x(k,l)k(\underline{n-k, m-l})$$

If the kernel were not **flipped**, the operation would correspond to **cross-correlation**, which is slightly different from convolution. Cross-correlation measures **similarity between two signals.**

2D-Convolutions

What if we want to keep the **same dimensions in output**?



2D-Convolutions + Padding

0	-1	0
-1	5	-1
0	-1	0

Kernel

1	1	3	2	0	1	1
1	1	3	2	0	1	1
2	2	1	0	1	3	3
1	1	3	2	1	1	1
2	2	0	2	3	2	2
1	1	3	1	2	1	1
1	1	3	1	2	1	1

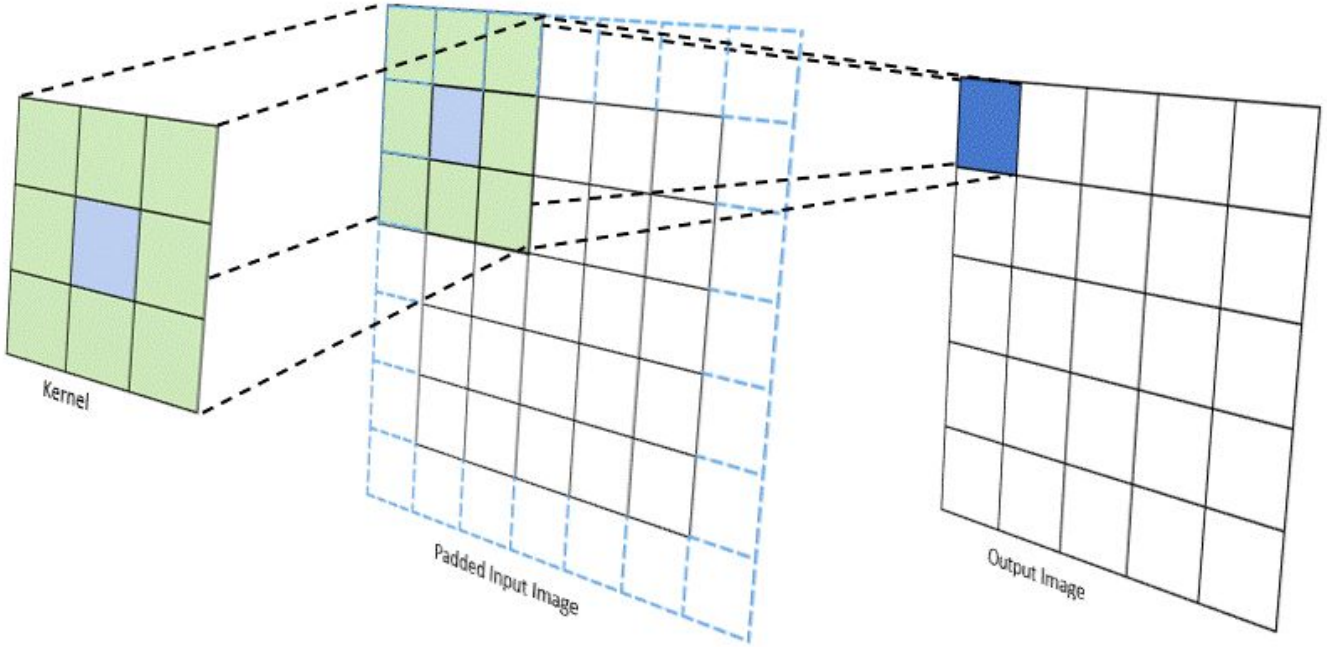
Padded Input Image

$$(0)(1) + (-1)(1) + (0)(3) + (-1)(1) + (5)(1) + (-1)(3) + (0)(2) + (-1)(2) + (0)(1) = -2$$

-2				

Output Image

2D-Convolutions + Padding



Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Mean Blur	$\begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix}$	

The identity kernel (a 3x3 matrix with 1 in the center and 0 elsewhere) l**eaves the image unchanged**. It simply multiplies each pixel by 1 and adds 0 to its neighbors, preserving the original image.

The sharpen kernel enhances edges by amplifying the difference between a pixel and its neighbors. The center value (5) boosts the current pixel, while the negative values around it subtract the neighbors.

The mean blur kernel averages each pixel with its 8 neighbors equally (each weighted 1/9). This smooths out differences between adjacent pixels, resulting in a blurred image where fine details are reduced.

Variation of the Mean blur

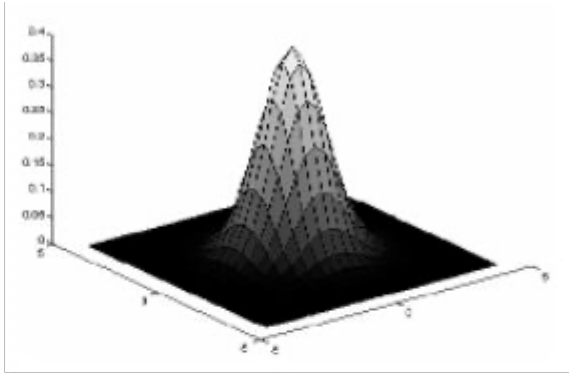
Gaussian Blur

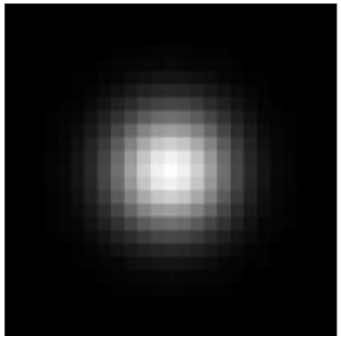
$$\begin{bmatrix} 1/16 & 2/16 & 1/16 \\ 2/16 & 4/16 & 2/16 \\ 1/16 & 2/16 & 1/16 \end{bmatrix}$$

Similar to mean blur, but weights pixels based on their distance from the center using a Gaussian distribution. This creates a more natural-looking blur, as it mimics the way our eyes perceive out-of-focus areas.

Proprieties

- Rotational Symmetric
- Weights nearby pixels more than distant ones



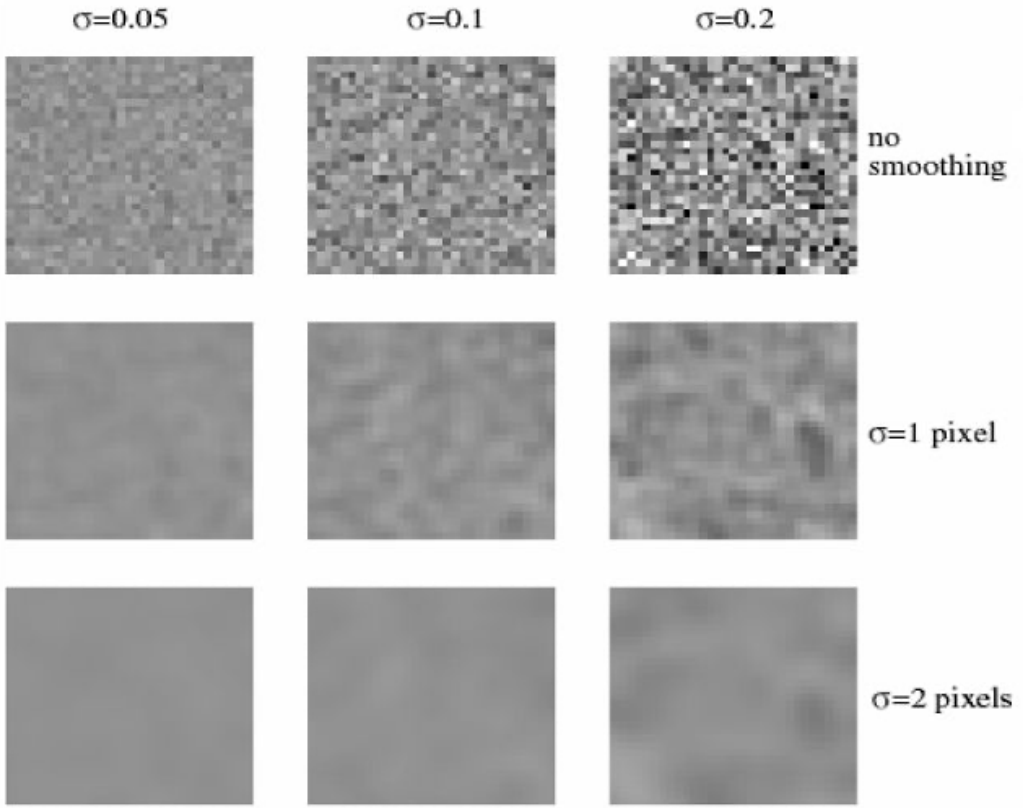


$$g(x,y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2+y^2}{2\sigma^2}\right)$$

Effects of smoothing:

- **each column** shows realizations of an image of Gaussian noise

- **each row** shows smoothing with Gaussians of different width



Separable property

- first convolve each row with a 1D filter
- then convolve each column with a 1D filter

$$f_x \otimes f_y = f_x \otimes f_y$$

$$\begin{bmatrix} \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \end{bmatrix} = \begin{bmatrix} \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \end{bmatrix} \otimes \begin{bmatrix} \frac{1}{3} \\ \frac{1}{3} \\ \frac{1}{3} \end{bmatrix}$$

References

- https://ai.stanford.edu/~syyeung/cvweb/tutorial1.html#:~:text=Image%20filtering%20changes%20the%20range,points%20without%20changing%20the%20colors
- https://medium.com/analytics-vidhya/a-beginners-guide-to-computer-vision-part-4-pyramid-3640edeffb00
- https://medium.com/@timothy_terati/image-convolution-filtering-a54dce7c786b
- https://docs.opencv.org/4.x/dc/dff/tutorial_py_pyramids.html